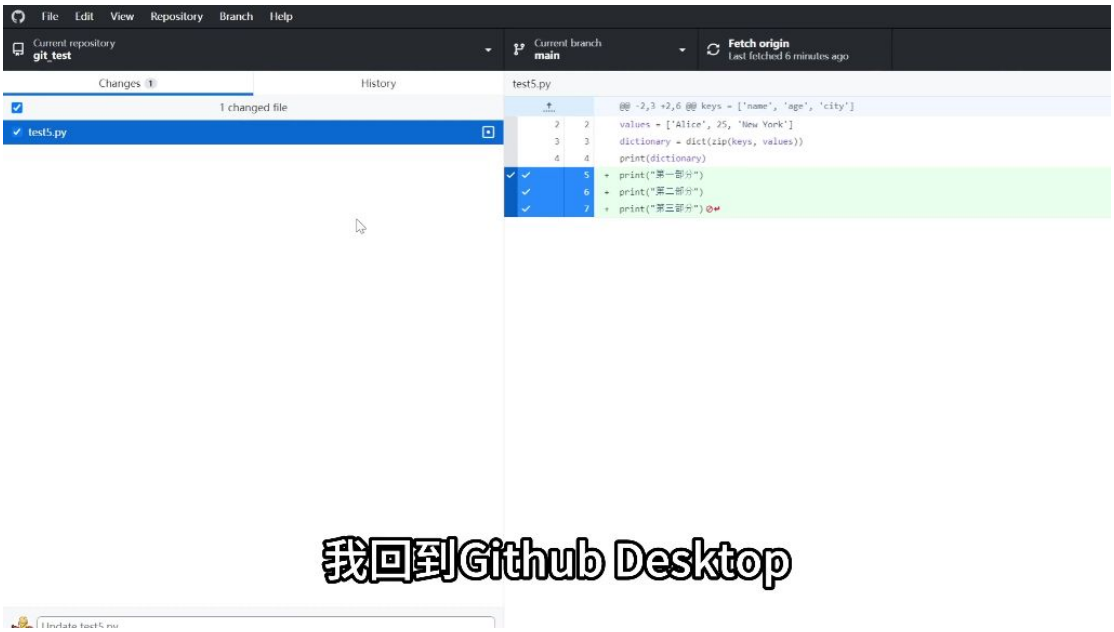
-- 以下为AI生成的图文笔记内容 --

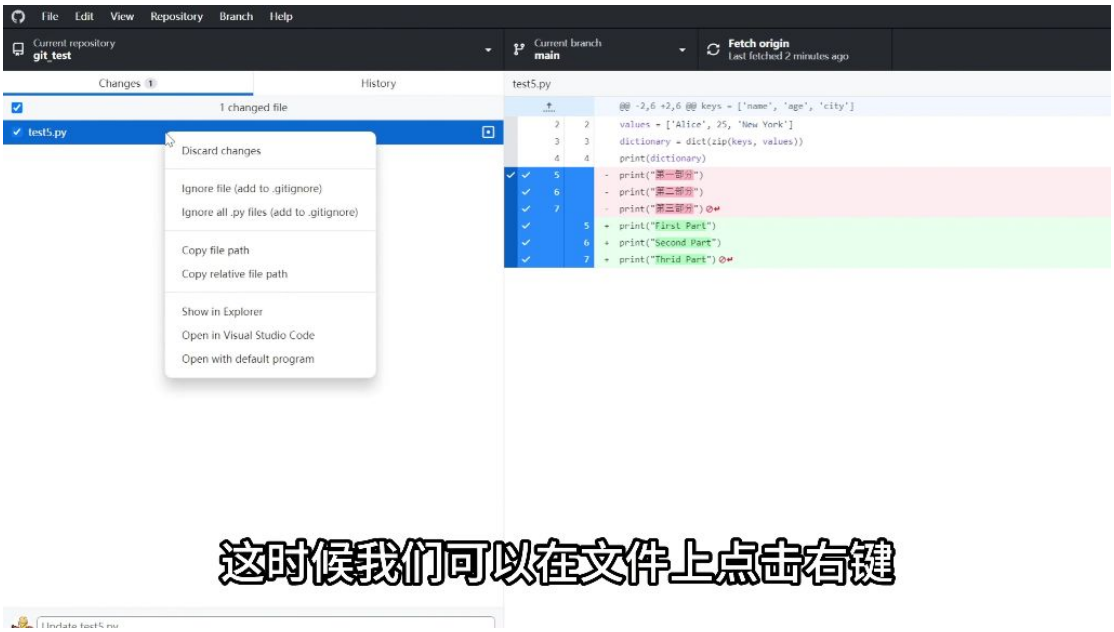
一、*GitHub Desktop进阶操作00:03*

1. 文件的部分提交 00:08



-
- **选择性提交**：可以只勾选需要提交的代码改动，未勾选的改动会保留在本地工作区
- **操作流程**：
  - 修改文件后保存
  - 在GitHub Desktop中取消勾选不需要提交的改动
  - 填写commit message后提交
  - 推送至远程仓库
- **应用场景**：当文件包含多个独立修改但只想提交部分内容时使用

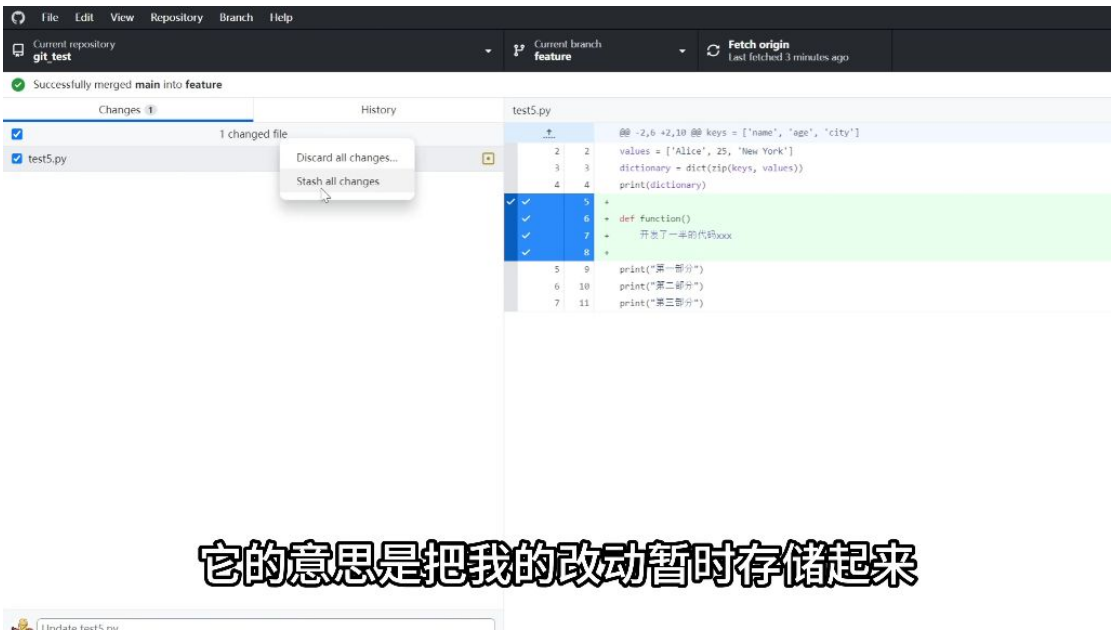
2. 放弃更改 01:26



这时候我们可以在文件上点击右键

-
- **操作方式**：右键文件 → Discard Changes
- **效果**：将文件还原到本地分支的最新状态
- **安全机制**：被放弃的文件会进入回收站，可恢复
- **注意事项**：此操作不可逆，建议先确认是否需要保留修改

3. 存储更改 02:06



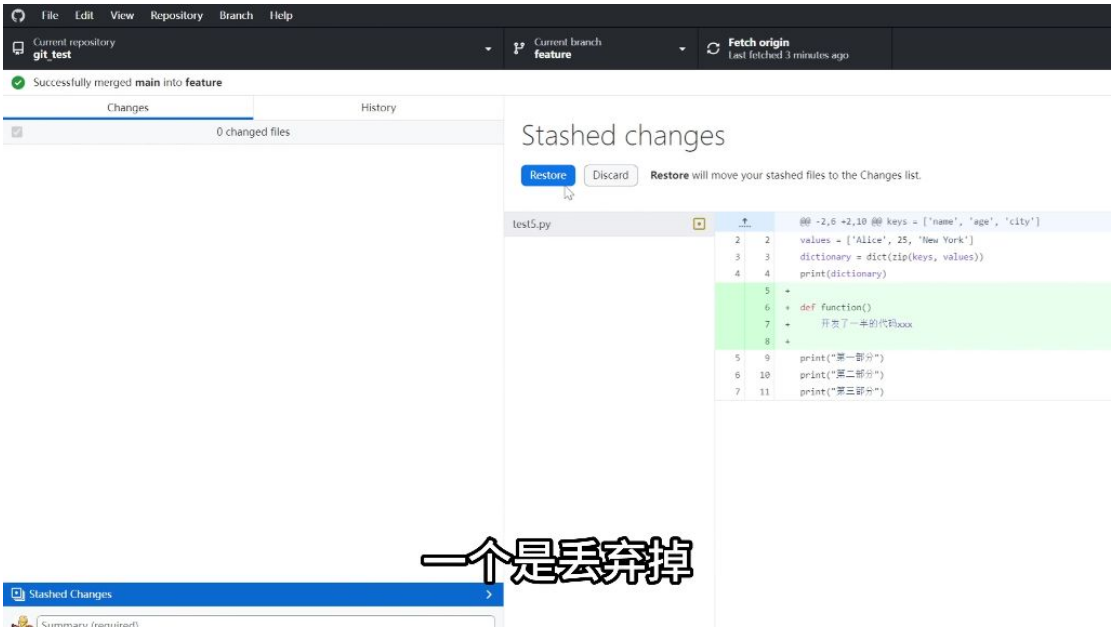
它的意思是把我的改动暂时存储起来

-
- **使用场景**：开发中途需要切换分支但不想提交未完成的工作
- **操作步骤**：
  - 右键更改的文件

- 选择"Stash All Changes"
- 系统会自动清空工作区更改

- **特点**：GitHub Desktop仅支持存储一次更改

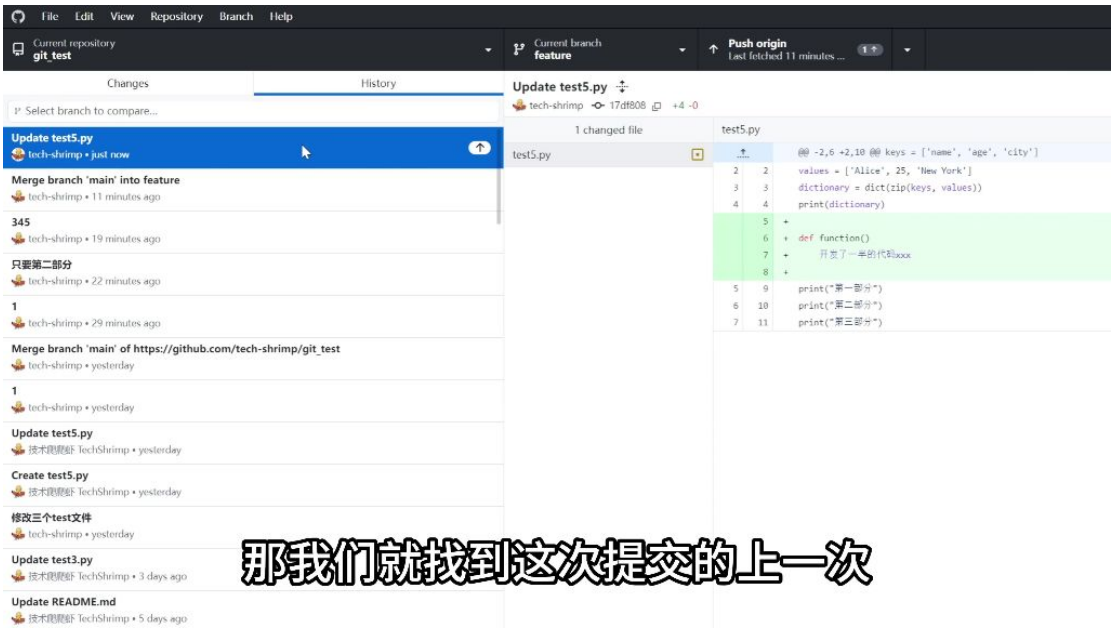
1）恢复存储的更改 02:40

- 

- **恢复方式**：
  - 在Changes标签底部找到Stashed Changes
  - 选择"Restore"恢复更改或"Discard"永久删除

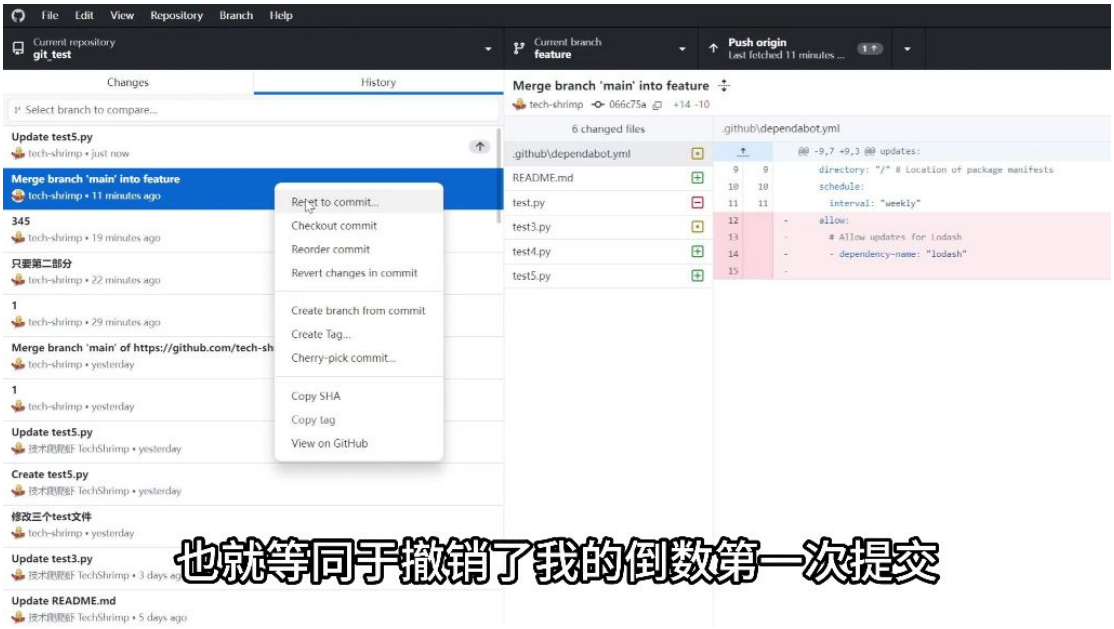
- **局限性**：无法像命令行那样存储多个stash

4. 撤回提交 03:53

- 

- **前提条件**：仅适用于未推送到远程的提交
- **操作步骤**：
  - 在History中找到目标提交
  - 右键选择"Reset to commit"
  - 选择reset方式（保留或丢弃工作区更改）

1）例题： 撤回提交操作 05:07

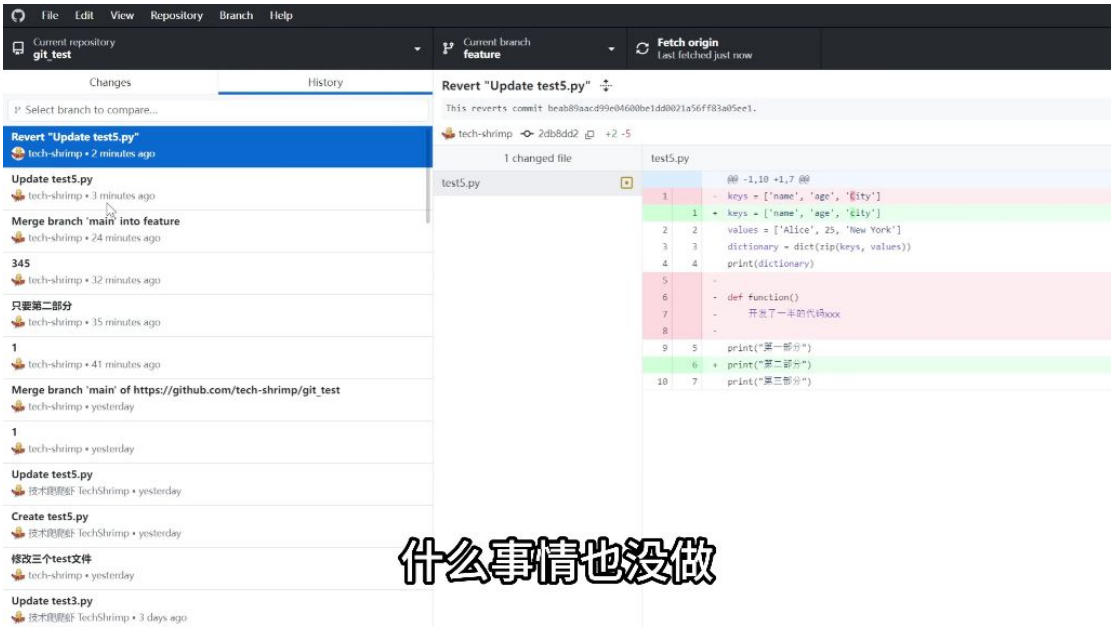
- 

- **命令行操作**：
  - git reset --mixed <commit ID>：撤回提交但保留更改

- ○ git push -f：强制推送（仅限个人分支使用）

- **注意事项**：强制推送会覆盖远程历史，禁止在协作分支使用

5. 反向操作 08:48

- 

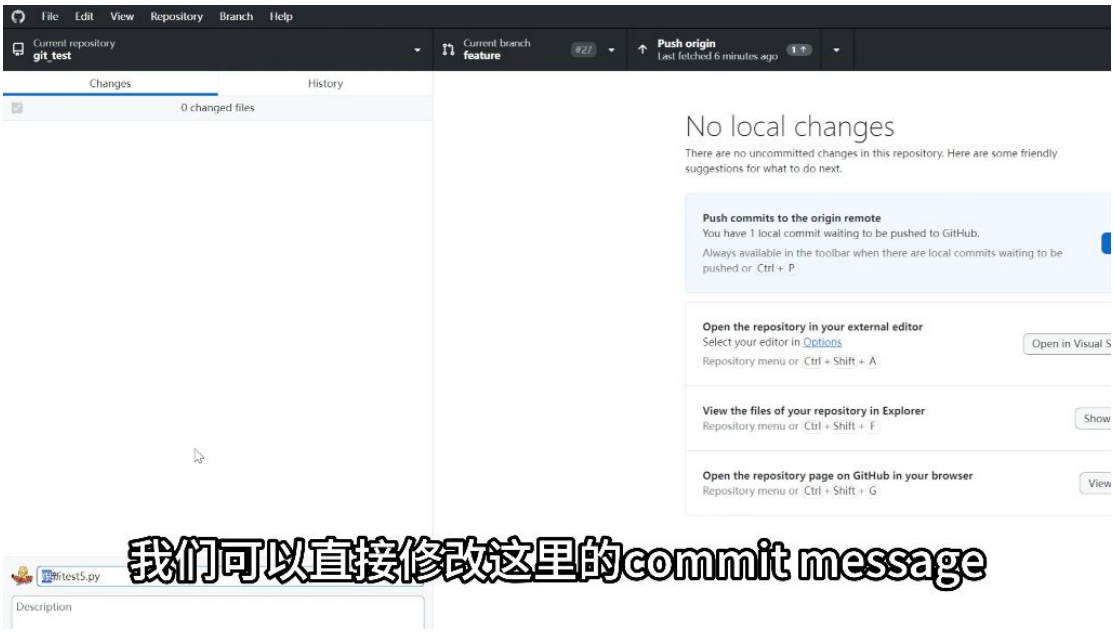
- **命令**：git revert <commit ID>

- **原理**：生成一个新提交来抵消指定提交的更改

- **优势**：不会修改历史记录，适合协作分支

- **效果**：新增的提交会反向执行原提交的所有修改

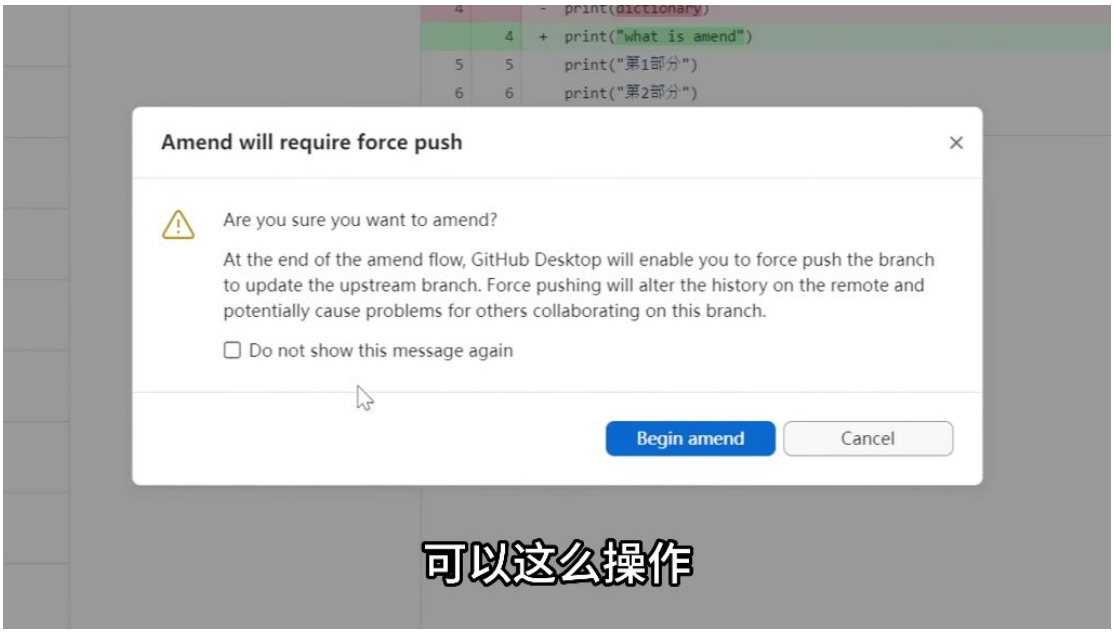
6. 修正提交 10:14

- 

- **适用范围**：仅能修改最新的一次提交
- **操作方式**：
  - 右键提交选择"Amend commit"
  - 修改commit message或文件内容
  - 确认修改
- **远程修正**：若已推送需强制推送，协作分支禁止使用

7. 应用案例 11:02

1）例题：提交修改修正

- 

- **典型场景**：修复拼写错误或补充遗漏的修改
- **风险提示**：强制推送会重写历史，可能导致协作问题
- **最佳实践**：小范围修改时使用，大修改建议新建提交

8. git后悔药总结 11:57

20. Git后悔药 /

git后悔药 0 / 0

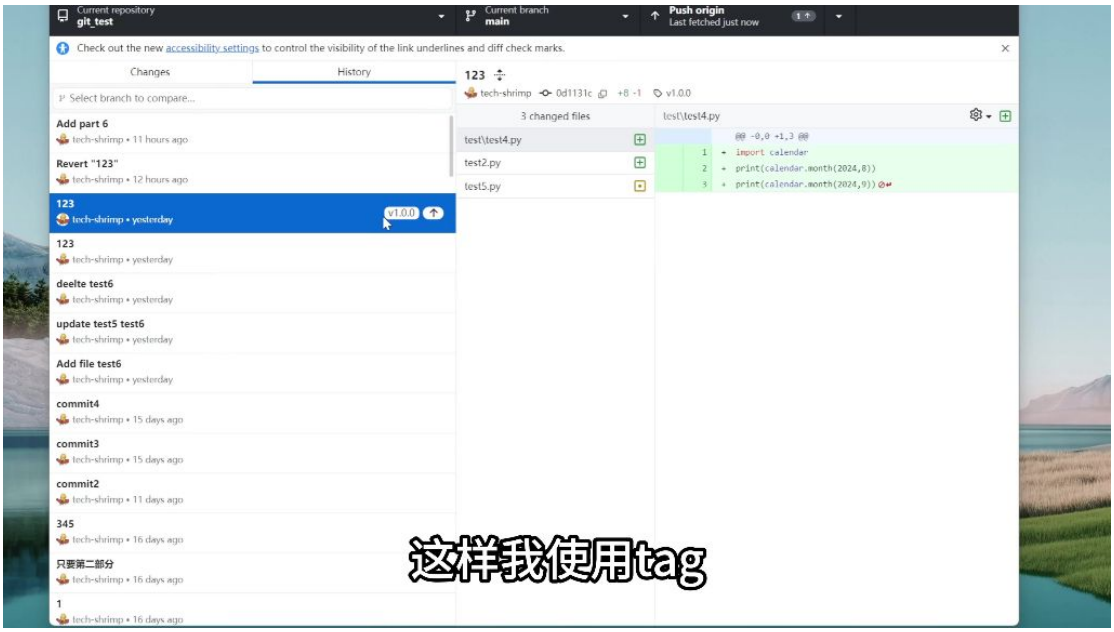
Git操作	Git 命令	使用场景	注意事项
discard	git restore <文件名> （**单个文件**） git reset --hard （**所有文件**）	工作区的修改还未commit	舍弃掉工作区修改的文件
reset	git reset <commit ID>	还原到某个commit的状态，舍弃之后的commit	如果reset已经推送远端的commit，会造成强制推送，集成分支禁止强推
revert	git revert <commit ID>	使用一个新的提交抵消掉某次commit的修改	
amend	git commit --amend	只能修改最新的一次commit	如果amend已经推送远端commit，会造成强制推送，集成分支禁止强推

对于单个文件 他对应的git命令是

-
- **discard**：
  - 命令：git restore <file>或git reset --hard
  - 场景：放弃未提交的修改
- **reset**：
  - 命令：git reset <commit ID>
  - 风险：强制推送会破坏协作
- **revert**：
  - 命令：git revert <commit ID>
  - 优势：安全不修改历史
- **amend**：
  - 命令：git commit --amend
  - 限制：仅限最新提交

9. 创建标签 13:18

1）例题：创建标签

- 

- **用途**：标记项目重要版本或里程碑
- **操作流程**：
  - 右键目标提交选择"Create tag"
  - 输入标签名（如v1.0.0）
  - 推送标签到远程
- **版本管理**：常用于软件版本发布标记

二、知识小结

知识点	核心内容	易混淆点/注意事项	操作风险等级
部分提交	通过取消勾选文件部分改动（如仅保留第二部分），实现选择性提交	未提交的改动仍保留在本地工作区	★（低风险）
放弃更改	右键文件选择Discard Changes，将文件还原为本地分支版本	被放弃的文件会进入回收站可恢复	★（低风险）
代码存储（Stash）	通过Stash All Changes临时存储未提交的改动，支持分支切换后恢复	GitHub Desktop仅支持单次存储（命令行无限制）	★★（中低风险）
强制切换分支	切换分支时选择Bring changes to [分支名]，	**不推荐**用于未完成的功能开发	★★★（中高风险）

	将未提交改动带入新分支		
Reset撤回提交	通过Reset to Commit撤销本地提交（保留文件更改）	**已推送的提交需强制推送（git push -f）**，禁止在协作分支使用	★★★★（高风险）
Revert反向提交	生成新提交抵消指定提交的改动（如新增变删除）	多人协作分支的安全撤回方案	★★（中低风险）
Amend修正提交	修改最新一次提交的代码或Message（需强制推送已推送的提交）	仅适用于最新提交，强制推送风险同Reset	★★★（中高风险）
Tag版本标记	为提交创建标签（如v1.0.0）标记版本里程碑	需手动Push Origin同步标签到远程	★（低风险）